Annex A Grammar

This clause is informative.

A.1 General

This annex contains the grammar productions found in the specification, including the optional ones for unsafe code. Productions appear here in the same order in which they appear in the specification.

A.2 Lexical grammar

ANTLR

```
// Source: §6.3.1 General
DEFAULT  : 'default' ;
NULL     : 'null' ;
TRUE     : 'true' ;
FALSE    : 'false' ;
ASTERISK : '*' ;
SLASH    : '/' ;

// Source: §6.3.1 General
input
    : input_section?
    ;

input_section
    : input_section_part+
    ;

input_section_part
    : input_element* New_Line
    | PP_Directive
    ;

input_element
    : Whitespace
    | Comment
    | token
```

```
    ;

// Source: §6.3.2 Line terminators
New_Line
    : New_Line_Character
    | '\u000D\u000A'    // carriage return, line feed
    ;

// Source: §6.3.3 Comments
Comment
    : Single_Line_Comment
    | Delimited_Comment
    ;

fragment Single_Line_Comment
    : '//' Input_Character*
    ;

fragment Input_Character
    // anything but New_Line_Character
    : ~('\u000D' | '\u000A'   | '\u0085' | '\u2028' | '\u2029')
    ;

fragment New_Line_Character
    : '\u000D'  // carriage return
    | '\u000A'  // line feed
    | '\u0085'  // next line
    | '\u2028'  // line separator
    | '\u2029'  // paragraph separator
    ;

fragment Delimited_Comment
    : '/*' Delimited_Comment_Section* ASTERISK+ '/'
    ;

fragment Delimited_Comment_Section
    : SLASH
    | ASTERISK* Not_Slash_Or_Asterisk
    ;

fragment Not_Slash_Or_Asterisk
    : ~('/' | '*')    // Any except SLASH or ASTERISK
    ;

// Source: §6.3.4 White space
Whitespace
    : [\p{Zs}]  // any character with Unicode class Zs
    | '\u0009'  // horizontal tab
    | '\u000B'  // vertical tab
```

```
    | '\u000C'  // form feed
    ;

// Source: §6.4.1 General
token
    : identifier
    | keyword
    | Integer_Literal
    | Real_Literal
    | Character_Literal
    | String_Literal
    | operator_or_punctuator
    ;

// Source: §6.4.2 Unicode character escape sequences
fragment Unicode_Escape_Sequence
    : '\\u' Hex_Digit Hex_Digit Hex_Digit Hex_Digit
    | '\\U' Hex_Digit Hex_Digit Hex_Digit Hex_Digit
            Hex_Digit Hex_Digit Hex_Digit Hex_Digit
    ;

// Source: §6.4.3 Identifiers
identifier
    : Simple_Identifier
    | contextual_keyword
    | discard_token
    ;

discard_token
    : '_'
    ;

Simple_Identifier
    : Available_Identifier
    | Escaped_Identifier
    ;

fragment Available_Identifier
    // excluding keywords or contextual keywords, see note below
    : Basic_Identifier
    ;

fragment Escaped_Identifier
    // Includes keywords and contextual keywords prefixed by '@'.
    // See note below.
    : '@' Basic_Identifier
    ;

fragment Basic_Identifier
```

```
    : Identifier_Start_Character Identifier_Part_Character*
    ;

fragment Identifier_Start_Character
    : Letter_Character
    | Underscore_Character
    ;

fragment Underscore_Character
    : '_'               // underscore
    | '\\u005' [fF]     // Unicode_Escape_Sequence for underscore
    | '\\U0000005' [fF] // Unicode_Escape_Sequence for underscore
    ;

fragment Identifier_Part_Character
    : Letter_Character
    | Decimal_Digit_Character
    | Connecting_Character
    | Combining_Character
    | Formatting_Character
    ;

fragment Letter_Character
    // Category Letter, all subcategories; category Number, subcate-
gory letter.
    : [\p{L}\p{Nl}]
    // Only escapes for categories L & Nl allowed. See note below.
    | Unicode_Escape_Sequence
    ;

fragment Combining_Character
    // Category Mark, subcategories non-spacing and spacing combin-
ing.
    : [\p{Mn}\p{Mc}]
    // Only escapes for categories Mn & Mc allowed. See note below.
    | Unicode_Escape_Sequence
    ;

fragment Decimal_Digit_Character
    // Category Number, subcategory decimal digit.
    : [\p{Nd}]
    // Only escapes for category Nd allowed. See note below.
    | Unicode_Escape_Sequence
    ;

fragment Connecting_Character
    // Category Punctuation, subcategory connector.
    : [\p{Pc}]
    // Only escapes for category Pc allowed. See note below.
```

```
    | Unicode_Escape_Sequence
    ;

fragment Formatting_Character
    // Category Other, subcategory format.
    : [\p{Cf}]
    // Only escapes for category Cf allowed, see note below.
    | Unicode_Escape_Sequence
    ;

// Source: §6.4.4 Keywords
keyword
    : 'abstract' | 'as'       | 'base'       | 'bool'      | 'break'
    | 'byte'     | 'case'     | 'catch'      | 'char'      |
'checked'
    | 'class'    | 'const'    | 'continue'   | 'decimal'   | DEFAULT
    | 'delegate' | 'do'       | 'double'     | 'else'      | 'enum'
    | 'event'    | 'explicit' | 'extern'     | FALSE       | 'file'
| 'finally'
    | 'fixed'    | 'float'    | 'for'        | 'foreach'   | 'goto'
    | 'if'       | 'implicit' | 'in'         | 'int'       | 'inter-
face'
    | 'internal' | 'is'       | 'lock'       | 'long'      | 'names-
pace'
    | 'new'      | NULL       | 'object'     | 'operator'  | 'out'
    | 'override' | 'params'   | 'private'    | 'protected' | 'pub-
lic'
    | 'readonly' | 'ref'      | 'return'     | 'sbyte'     |
'sealed'
    | 'short'    | 'sizeof'   | 'stackalloc' | 'static'    |
'string'
    | 'struct'   | 'switch'   | 'this'       | 'throw'     | TRUE
    | 'try'      | 'typeof'   | 'uint'       | 'ulong'     |
'unchecked'
    | 'unsafe'   | 'ushort'   | 'using'      | 'virtual'   | 'void'
    | 'volatile' | 'while'
    ;

// Source: §6.4.4 Keywords
contextual_keyword
    : 'add'      | 'alias'      | 'and'        | 'ascending' |
'async'
    | 'await'    | 'by'         | 'Cdecl'      | 'descending'| 'dy-
namic'
    | 'equals'   | 'Fastcall'   | 'from'       | 'get'       |
'global'
    | 'group'    | 'init'       | 'into'       | 'join'      | 'let'
    | 'managed'  | 'nameof'     | 'nint'       | 'not'       | 'not-
null'
```

```
    | 'nuint'    | 'on'         | 'or'         | 'orderby'    | 'par-
tial'
    | 'record'   | 'remove'     | 'required'   | 'scoped'     | 'se-
lect'    | 'set'        | 'Stdcall'
    | 'Thiscall' | 'unmanaged'  | 'value'      | 'var'        |
'when'
    | 'where'    | 'yield'
    ;

// Source: §6.4.5.1 General
literal
    : boolean_literal
    | Integer_Literal
    | Real_Literal
    | Character_Literal
    | String_Literal
    | null_literal
    ;

// Source: §6.4.5.2 Boolean literals
boolean_literal
    : TRUE
    | FALSE
    ;

// Source: §6.4.5.3 Integer literals
Integer_Literal
    : Decimal_Integer_Literal
    | Hexadecimal_Integer_Literal
    | Binary_Integer_Literal
    ;

fragment Decimal_Integer_Literal
    : Decimal_Digit Decorated_Decimal_Digit* Integer_Type_Suffix?
    ;

fragment Decorated_Decimal_Digit
    : '_'* Decimal_Digit
    ;

fragment Decimal_Digit
    : '0'..'9'
    ;

fragment Integer_Type_Suffix
    : 'U' | 'u' | 'L' | 'l' |
      'UL' | 'Ul' | 'uL' | 'ul' | 'LU' | 'Lu' | 'lU' | 'lu'
    ;
```

```
fragment Hexadecimal_Integer_Literal
    : ('0x' | '0X') Decorated_Hex_Digit+ Integer_Type_Suffix?
    ;

fragment Decorated_Hex_Digit
    : '_'* Hex_Digit
    ;

fragment Hex_Digit
    : '0'..'9' | 'A'..'F' | 'a'..'f'
    ;

fragment Binary_Integer_Literal
    : ('0b' | '0B') Decorated_Binary_Digit+ Integer_Type_Suffix?
    ;

fragment Decorated_Binary_Digit
    : '_'* Binary_Digit
    ;

fragment Binary_Digit
    : '0' | '1'
    ;

// Source: §6.4.5.4 Real literals
Real_Literal
    : Decimal_Digit Decorated_Decimal_Digit* '.'
      Decimal_Digit Decorated_Decimal_Digit* Exponent_Part?
Real_Type_Suffix?
    | '.' Decimal_Digit Decorated_Decimal_Digit* Exponent_Part?
Real_Type_Suffix?
    | Decimal_Digit Decorated_Decimal_Digit* Exponent_Part
Real_Type_Suffix?
    | Decimal_Digit Decorated_Decimal_Digit* Real_Type_Suffix
    ;

fragment Exponent_Part
    : ('e' | 'E') Sign? Decimal_Digit Decorated_Decimal_Digit*
    ;

fragment Sign
    : '+' | '-'
    ;

fragment Real_Type_Suffix
    : 'F' | 'f' | 'D' | 'd' | 'M' | 'm'
    ;

// Source: §6.4.5.5 Character literals
```

```
Character_Literal
    : '\'' Character '\''
    ;

fragment Character
    : Single_Character
    | Simple_Escape_Sequence
    | Hexadecimal_Escape_Sequence
    | Unicode_Escape_Sequence
    ;

fragment Single_Character
    // anything but ', \, and New_Line_Character
    : ~['\\\u000D\u000A\u0085\u2028\u2029]
    ;

fragment Simple_Escape_Sequence
    : '\\\'' | '\\"' | '\\\\' | '\\0' | '\\a' | '\\b' |
      '\\f' | '\\n' | '\\r' | '\\t' | '\\v'
    ;

fragment Hexadecimal_Escape_Sequence
    : '\\x' Hex_Digit Hex_Digit? Hex_Digit? Hex_Digit?
    ;

// Source: §6.4.5.6 String literals
String_Literal
    : Regular_String_Literal
    | Verbatim_String_Literal
    | Raw_String_Literal
    ;

fragment Regular_String_Literal
    : '"' Regular_String_Literal_Character* '"' Utf8_Suffix?
    ;

fragment Regular_String_Literal_Character
    : Single_Regular_String_Literal_Character
    | Simple_Escape_Sequence
    | Hexadecimal_Escape_Sequence
    | Unicode_Escape_Sequence
    ;

fragment Single_Regular_String_Literal_Character
    // anything but ", \, and New_Line_Character
    : ~["\\\u000D\u000A\u0085\u2028\u2029]
    ;

fragment Verbatim_String_Literal
```

```
    : '@"' Verbatim_String_Literal_Character* '"' Utf8_Suffix?
    ;

fragment Verbatim_String_Literal_Character
    : Single_Verbatim_String_Literal_Character
    | Quote_Escape_Sequence
    ;

fragment Single_Verbatim_String_Literal_Character
    : ~["]     // anything but quotation mark (U+0022)
    ;

fragment Quote_Escape_Sequence
    : '""'
    ;

fragment Raw_String_Literal
    : Single_Line_Raw_String_Literal
    | Multi_Line_Raw_String_Literal
    ;

fragment Single_Line_Raw_String_Literal
    : Raw_String_Literal_Delimiter  Raw_String_Literal Content
      Raw_String_Literal_Delimiter
    ;

fragment Raw_String_Literal_Delimiter
    : '"""'  '"'*
    ;

fragment Raw_String_Literal Content
    // anything except New_Line
    : ~( '\u000D\u000A' | '\u000D' | '\u000A' | '\u0085' | '\u2028'
| '\u2029')
    ;

fragment Multi_Line_Raw_String_Literal
    : Raw_String_Literal_Delimiter Whitespace* New_Line
      (Raw_String_Literal Content | New_Line)* New_Line
      Whitespace* Raw_String_Literal_Delimiter
    ;

fragment Utf8_Suffix
    : 'u8' | 'U8'
    ;

// Source: §6.4.5.7 The null literal
null_literal
    : NULL
```

```
    ;

// Source: §6.4.6 Operators and punctuators
operator_or_punctuator
    : '{'  | '}'  | '['  | ']'  | '('  | ')'  | '.'  | ','  | ':'
| ';'
    | '+'  | '-'  | '*'  | '/'  | '%'  | '&'  | '|'  | '^'  | '!'
| '~'
    | '='  | '<'  | '>'  | '?'  | '??'  | '::' | '++'  | '--' | '&&'
| '||'
    | '->' | '==' | '!=' | '<=' | '>='  | '+=' | '-='  | '*=' | '/='
| '%='
    | '&=' | '|=' | '^=' | '<<' | '<<=' | '=>' | '??=' | '..'
    ;

right_shift
    : '>'  '>'
    ;

right_shift_assignment
    : '>' '>='
    ;

unsigned_right_shift
    : '>'  '>'  '>'
    ;

unsigned_right_shift_assignment
    : '>'  '>'  '>='
    ;

// Source: §6.5.1 General
PP_Directive
    : PP_Start PP_Kind PP_New_Line
    ;

fragment PP_Kind
    : PP_Declaration
    | PP_Conditional
    | PP_Line
    | PP_Diagnostic
    | PP_Region
    | PP_Pragma
    | PP_Nullable
    ;

// Only recognised at the beginning of a line
fragment PP_Start
    // See note below.
```

```
    : { getCharPositionInLine() == 0 }? PP_Whitespace? '#'
PP_Whitespace?
    ;

fragment PP_Whitespace
    : ( [\p{Zs}]  // any character with Unicode class Zs
      | '\u0009'  // horizontal tab
      | '\u000B'  // vertical tab
      | '\u000C'  // form feed
      )+
    ;

fragment PP_New_Line
    : PP_Whitespace? Single_Line_Comment? New_Line
    ;

// Source: §6.5.2 Conditional compilation symbols
fragment PP_Conditional_Symbol
    // Must not be equal to tokens TRUE or FALSE. See note below.
    : Basic_Identifier
    ;

// Source: §6.5.3 Pre-processing expressions
fragment PP_Expression
    : PP_Whitespace? PP_Or_Expression PP_Whitespace?
    ;

fragment PP_Or_Expression
    : PP_And_Expression (PP_Whitespace? '||' PP_Whitespace?
PP_And_Expression)*
    ;

fragment PP_And_Expression
    : PP_Equality_Expression (PP_Whitespace? '&&' PP_Whitespace?
      PP_Equality_Expression)*
    ;

fragment PP_Equality_Expression
    : PP_Unary_Expression (PP_Whitespace? ('==' | '!=')
PP_Whitespace?
      PP_Unary_Expression)*
    ;

fragment PP_Unary_Expression
    : PP_Primary_Expression
    | '!' PP_Whitespace? PP_Unary_Expression
    ;

fragment PP_Primary_Expression
```

```
    : TRUE
    | FALSE
    | PP_Conditional_Symbol
    | '(' PP_Whitespace? PP_Expression PP_Whitespace? ')'
    ;

// Source: §6.5.4 Definition directives
fragment PP_Declaration
    : 'define' PP_Whitespace PP_Conditional_Symbol
    | 'undef' PP_Whitespace PP_Conditional_Symbol
    ;

// Source: §6.5.5 Conditional compilation directives
fragment PP_Conditional
    : PP_If_Section
    | PP_Elif_Section
    | PP_Else_Section
    | PP_Endif
    ;

fragment PP_If_Section
    : 'if' PP_Whitespace PP_Expression
    ;

fragment PP_Elif_Section
    : 'elif' PP_Whitespace PP_Expression
    ;

fragment PP_Else_Section
    : 'else'
    ;

fragment PP_Endif
    : 'endif'
    ;

// Source: §6.5.6 Diagnostic directives
fragment PP_Diagnostic
    : 'error' PP_Message?
    | 'warning' PP_Message?
    ;

fragment PP_Message
    : PP_Whitespace Input_Character*
    ;

// Source: §6.5.7 Region directives
fragment PP_Region
    : PP_Start_Region
```

```
    | PP_End_Region
    ;

fragment PP_Start_Region
    : 'region' PP_Message?
    ;

fragment PP_End_Region
    : 'endregion' PP_Message?
    ;

// Source: §6.5.8 Line directives
fragment PP_Line
    : 'line' PP_Whitespace PP_Line_Indicator
    ;

fragment PP_Line_Indicator
    : Decimal_Digit+ PP_Whitespace PP_Compilation_Unit_Name
    | Decimal_Digit+
    | DEFAULT
    | 'hidden'
    | PP_Start_Line_Character PP_Whitespace? '-' PP_Whitespace?
PP_End_Line_Character
      PP_Whitespace (PP_Character_Offset PP_Whitespace)?
PP_Compilation_Unit_Name
    ;

fragment PP_Compilation_Unit_Name
    : '"' PP_Compilation_Unit_Name_Character* '"'
    ;

fragment PP_Compilation_Unit_Name_Character
    // Any Input_Character except "
    : ~('\u000D' | '\u000A'   | '\u0085' | '\u2028' | '\u2029' |
'"')
    ;

fragment PP_Start_Line_Character
    : '(' PP_Whitespace? PP_Start_Line PP_Whitespace? ','
PP_Whitespace?
      PP_Start_Character PP_Whitespace? ')'
    ;
fragment PP_End_Line_Character
    : '(' PP_Whitespace? PP_End_Line PP_Whitespace? ','
PP_Whitespace?
      PP_End_Character PP_Whitespace? ')'
    ;

fragment PP_Start_Line
```

```
    : Decimal_Digit+
    ;

fragment PP_End_Line
    : Decimal_Digit+
    ;

fragment PP_Start_Character
    : Decimal_Digit+
    ;

fragment PP_End_Character
    : Decimal_Digit+
    ;

fragment PP_Character_Offset
    : Decimal_Digit+
    ;

// Source: §6.5.9 Nullable directive
fragment PP_Nullable
    : 'nullable' PP_Whitespace PP_Nullable_Action
      (PP_Whitespace PP_Nullable_Target)?
    ;
fragment PP_Nullable_Action
    : 'disable'
    | 'enable'
    | 'restore'
    ;
fragment PP_Nullable_Target
    : 'warnings'
    | 'annotations'
    ;

// Source: §6.5.10 Pragma directives
fragment PP_Pragma
    : 'pragma' PP_Pragma_Text?
    ;

fragment PP_Pragma_Text
    : PP_Whitespace Input_Character*
    ;
```

A.3 Syntactic grammar

ANTLR

```
// Source: §7.8.1 General
namespace_name
    : namespace_or_type_name
    ;

type_name
    : namespace_or_type_name
    ;

namespace_or_type_name
    : identifier type_argument_list? ('.' identifier type_argu-
ment_list?)*
    | qualified_alias_member ('.' identifier type_argument_list?)*
    ;

// Source: §8.1 General
type
    : reference_type
    | value_type
    | type_parameter
    | pointer_type     // unsafe code support
    ;

// Source: §8.2.1 General
reference_type
    : non_nullable_reference_type
    | nullable_reference_type
    ;

non_nullable_reference_type
    : class_type
    | interface_type
    | array_type
    | delegate_type
    | 'dynamic'
    ;

class_type
    : type_name
    | 'object'
    | 'string'
    ;

interface_type
    : type_name
    ;
```

```
array_type
    : array_type nullable_type_annotation rank_specifier+
    | non_array_type rank_specifier+
    ;

non_array_type
    : non_array_non_nullable_type nullable_type_annotation?
    | pointer_type      // unsafe code support
    ;

non_array_non_nullable_type
    : non_nullable_value_type
    | class_type
    | interface_type
    | delegate_type
    | 'dynamic'
    | type_parameter
    ;

rank_specifier
    : '[' ','* ']'
    ;

delegate_type
    : type_name
    ;

nullable_reference_type
    : non_nullable_reference_type nullable_type_annotation
    ;

nullable_type_annotation
    : '?'
    ;


// Source: §8.3.1 General
value_type
    : non_nullable_value_type
    | nullable_value_type
    ;

non_nullable_value_type
    : struct_type
    | enum_type
    ;

struct_type
    : type_name
```

```
    | simple_type
    | tuple_type
    ;

simple_type
    : numeric_type
    | 'bool'
    ;

numeric_type
    : integral_type
    | floating_point_type
    | 'decimal'
    ;

integral_type
    : 'sbyte'
    | 'byte'
    | 'short'
    | 'ushort'
    | 'int'
    | 'uint'
    | 'nint'
    | 'nuint'
    | 'long'
    | 'ulong'
    | 'char'
    ;

floating_point_type
    : 'float'
    | 'double'
    ;

tuple_type
    : '(' tuple_type_element (',' tuple_type_element)+ ')'
    ;

tuple_type_element
    : type identifier?
    ;

enum_type
    : type_name
    ;

nullable_value_type
    : non_nullable_value_type nullable_type_annotation
    ;
```

```
// Source: §8.4.2 Type arguments
type_argument_list
    : '<' type_argument (',' type_argument)* '>'
    ;

type_argument
    : type
    | type_parameter nullable_type_annotation?
    ;

// Source: §8.5 Type parameters
type_parameter
    : identifier
    ;

// Source: §8.8 Unmanaged types
unmanaged_type
    : value_type
    | pointer_type     // unsafe code support
    ;

// Source: §9.5 Variable references
variable_reference
    : expression
    ;

// Source: §11.2.1 General
pattern
    : logical_pattern
    ;

primary_pattern
    : parenthesized_pattern
    | declaration_pattern
    | constant_pattern
    | var_pattern
    | positional_pattern
    | property_pattern
    | discard_pattern
    | type_pattern
    | relational_pattern
    | logical_pattern
    | list_pattern
    | slice_pattern
    ;

// Source: §11.2.2 Declaration pattern
declaration_pattern
```

```
    : type simple_designation
    ;
simple_designation
    : discard_designation
    | single_variable_designation
    ;
discard_designation
    : '_'
    ;
single_variable_designation
    : identifier
    ;

// Source: §11.2.3 Constant pattern
constant_pattern
    : constant_expression
    ;

// Source: §11.2.4 Var pattern
var_pattern
    : 'var' designation
    ;
designation
    : simple_designation
    | tuple_designation
    ;
tuple_designation
    : '(' designations? ')'
    ;
designations
    : designation (',' designation)*
    ;

// Source: §11.2.5 Positional pattern
positional_pattern
    : type? '(' subpatterns? ')' property_subpattern? simple_desig-
nation?
    ;
subpatterns
    : subpattern (',' subpattern)*
    ;
subpattern
    : pattern
    | subpattern_name ':' pattern
    ;
subpattern_name
    : identifier
    | subpattern_name '.' identifier
    ;
```

```
// Source: §11.2.6 Property pattern
property_pattern
    : type? property_subpattern simple_designation?
    ;
property_subpattern
    : '{' '}'
    | '{' subpatterns ','? '}'
    ;

// Source: §11.2.7 Discard pattern
discard_pattern
    : '_'
    ;

// Source: §11.2.8 Type pattern
type_pattern
    : type
    ;

// Source: §11.2.9 Relational pattern
relational_pattern
    : '<'  relational_expression
    | '<=' relational_expression
    | '>'  relational_expression
    | '>=' relational_expression
    ;

// Source: §11.2.10 Logical pattern
logical_pattern
    : disjunctive_pattern
    ;

disjunctive_pattern
    : disjunctive_pattern 'or' conjunctive_pattern
    | conjunctive_pattern
    ;

conjunctive_pattern
    : conjunctive_pattern 'and' negated_pattern
    | negated_pattern
    ;

negated_pattern
    : 'not' negated_pattern
    | primary_pattern
    ;

// Source: §11.2.11 List pattern
```

```
list_pattern
    : list_pattern_clause simple_designation?
    ;

list_pattern_clause
    : '[' (pattern (',' pattern)* ','?)? ']'
    ;

// Source: §11.2.12 Slice pattern
slice_pattern
    : '..' pattern?
    ;

// Source: §12.6.2.1 General
argument_list
    : argument (',' argument)*
    ;

argument
    : argument_name? argument_value
    ;

argument_name
    : identifier ':'
    ;

argument_value
    : expression
    | 'in' 'scoped'? variable_reference
    | 'ref' 'scoped'? variable_reference
    | 'out' 'scoped'? declaration_expression
    | 'out' 'scoped'? variable_reference
    ;

// Source: §12.8.1 General
primary_expression
    : literal
    | interpolated_string_expression
    | simple_name
    | parenthesized_expression
    | tuple_literal
    | member_access
    | null_conditional_member_access
    | invocation_expression
    | element_access
    | null_conditional_element_access
    | this_access
    | base_access
    | post_increment_expression
```

```
    | post_decrement_expression
    | null_forgiving_expression
    | array_creation_expression
    | object_creation_expression
    | delegate_creation_expression
    | anonymous_object_creation_expression
    | typeof_expression
    | sizeof_expression
    | checked_expression
    | unchecked_expression
    | default_value_expression
    | nameof_expression
    | anonymous_method_expression
    | pointer_member_access     // unsafe code support
    | pointer_element_access    // unsafe code support
    | stackalloc_expression
    ;

// Source: §12.8.3 Interpolated string expressions
interpolated_string_expression
    : interpolated_regular_string_expression
    | interpolated_verbatim_string_expression
    | interpolated_raw_string_expression
    ;

// interpolated regular string expressions

interpolated_regular_string_expression
    : Interpolated_Regular_String_Start
Interpolated_Regular_String_Mid?
      ('{' regular_interpolation '}'
Interpolated_Regular_String_Mid?)*
      Interpolated_Regular_String_End
    ;

regular_interpolation
    : expression (',' interpolation_minimum_width)?
      Regular_Interpolation_Format?
    ;

interpolation_minimum_width
    : constant_expression
    ;

Interpolated_Regular_String_Start
    : '$"'
    ;

// the following three lexical rules are context sensitive, see de-
```

```
tails below

Interpolated_Regular_String_Mid
    : Interpolated_Regular_String_Element+
    ;

Regular_Interpolation_Format
    : ':' Interpolated_Regular_String_Element+
    ;

Interpolated_Regular_String_End
    : '"'
    ;

fragment Interpolated_Regular_String_Element
    : Interpolated_Regular_String_Character
    | Simple_Escape_Sequence
    | Hexadecimal_Escape_Sequence
    | Unicode_Escape_Sequence
    | Open_Brace_Escape_Sequence
    | Close_Brace_Escape_Sequence
    ;

fragment Interpolated_Regular_String_Character
    // Any character except " (U+0022), \\ (U+005C),
    // { (U+007B), and } (U+007D).
    : ~["\\{}]
    ;

// interpolated verbatim string expressions

interpolated_verbatim_string_expression
    : Interpolated_Verbatim_String_Start
Interpolated_Verbatim_String_Mid?
      ('{' verbatim_interpolation '}'
Interpolated_Verbatim_String_Mid?)*
      Interpolated_Verbatim_String_End
    ;

verbatim_interpolation
    : expression (',' interpolation_minimum_width)?
      Verbatim_Interpolation_Format?
    ;

Interpolated_Verbatim_String_Start
    : '$@"'
    | '@$"'
    ;
```

```
// the following three lexical rules are context sensitive, see de-
tails below

Interpolated_Verbatim_String_Mid
    : Interpolated_Verbatim_String_Element+
    ;

Verbatim_Interpolation_Format
    : ':' Interpolated_Verbatim_String_Element+
    ;

Interpolated_Verbatim_String_End
    : '"'
    ;

fragment Interpolated_Verbatim_String_Element
    : Interpolated_Verbatim_String_Character
    | Quote_Escape_Sequence
    | Open_Brace_Escape_Sequence
    | Close_Brace_Escape_Sequence
    ;

fragment Interpolated_Verbatim_String_Character
    : ~["{}]    // Any character except " (U+0022), { (U+007B) and }
(U+007D)
    ;

// lexical fragments used by both regular and verbatim interpolated
strings

fragment Open_Brace_Escape_Sequence
    : '{{'
    ;

fragment Close_Brace_Escape_Sequence
    : '}}'
    ;

// interpolated raw string expressions

interpolated_raw_string_expression
    : single_line_interpolated_raw_string_expression
    | multi_line_interpolated_raw_string_expression
    ;

single_line_interpolated_raw_string_expression
    : Interpolated_Raw_String_Start  Interpolated_Raw_String_Mid
        Interpolated_Raw_String_End
    ;
```

```
Interpolated_Raw_String_Prefix
    : '$'+
    ;

Interpolated_Raw_String_Start
    : Interpolated_Raw_String_Prefix  Raw_String_Literal_Delimiter
    ;

// the following two lexical rules are context sensitive, see de-
tails below

Interpolated_Raw_String_Mid
    : (Raw_String_Literal_Content | raw_interpolation)+
    ;

Interpolated_Raw_String_End
    : Raw_String_Literal_Delimiter
    ;

raw_interpolation
    : raw_interpolation_start expression
      (',' interpolation_minimum_width)? Raw_Interpolation_Format?
      raw_interpolation_end
    ;

raw_interpolation_start
    : '{'+
    ;

raw_interpolation_end
    : '}'+
    ;

// the following lexical rule is context sensitive, see details be-
low

Raw_Interpolation_Format
    : ':' Interpolated_Raw_String_Character+
;

fragment Interpolated_Raw_String_Character
    // Any character except " (U+0022), \\ (U+005C),
    // { (U+007B), } (U+007D), and New_Line_Character.
    : ~["\\{}\u000D\u000A\u0085\u2028\u2029]
    ;

multi_line_interpolated_raw_string_expression
    : Interpolated_Raw_String_Start Whitespace* New_Line
```

```
      (Interpolated_Raw_String_Mid | New_Line)* New_Line
      Whitespace* Interpolated_Raw_String_End
    ;

// Source: §12.8.4 Simple names
simple_name
    : identifier type_argument_list?
    ;

// Source: §12.8.5 Parenthesized expressions
parenthesized_expression
    : '(' expression ')'
    ;

// Source: §12.8.6 Tuple literals
tuple_literal
    : '(' tuple_element (',' tuple_element)+ ')'
    ;

tuple_element
    : (identifier ':')? expression
    ;

// Source: §12.8.7.1 General
member_access
    : primary_expression '.' identifier type_argument_list?
    | predefined_type '.' identifier type_argument_list?
    | qualified_alias_member '.' identifier type_argument_list?
    ;

predefined_type
    : 'bool' | 'byte' | 'char' | 'decimal' | 'double' | 'float' |
'int'
    | 'long' | 'object' | 'sbyte' | 'short' | 'string' | 'uint' |
'ulong'
    | 'ushort'
    ;

// Source: §12.8.8 Null Conditional Member Access
null_conditional_member_access
    : primary_expression '?' '.' identifier type_argument_list?
      (null_forgiving_operator? dependent_access)*
    ;

dependent_access
    : '.' identifier type_argument_list?    // member access
    | '[' argument_list ']'                 // element access
    | '(' argument_list? ')'                // invocation
    ;
```

```
null_conditional_projection_initializer
    : primary_expression '?' '.' identifier type_argument_list?
    ;

// Source: §12.8.9.1 General
null_forgiving_expression
    : primary_expression null_forgiving_operator
    ;

null_forgiving_operator
    : '!'
    ;

// Source: §12.8.10.1 General
invocation_expression
    : primary_expression '(' argument_list? ')'
    ;

// Source: §12.8.11 Null Conditional Invocation Expression
null_conditional_invocation_expression
    : null_conditional_member_access null_forgiving_operator? '('
argument_list? ')'
    | null_conditional_element_access null_forgiving_operator? '('
argument_list? ')'
    ;

// Source: §12.8.12.1 General
element_access
    : primary_expression '[' argument_list ']'
    ;

// Source: §12.8.13 Null Conditional Element Access
null_conditional_element_access
    : primary_expression '?' '[' argument_list ']'
      (null_forgiving_operator? dependent_access)*
    ;

// Source: §12.8.14 This access
this_access
    : 'this'
    ;

// Source: §12.8.15 Base access
base_access
    : 'base' '.' identifier type_argument_list?
    | 'base' '[' argument_list ']'
    ;
```

```
// Source: §12.8.16 Postfix increment and decrement operators
post_increment_expression
    : primary_expression '++'
    ;

post_decrement_expression
    : primary_expression '--'
    ;

// Source: §12.8.17.2.1 General
object_creation_expression
    : 'new' type '(' argument_list? ')' object_or_collection_ini-
tializer?
    | 'new' type object_or_collection_initializer
    | target_typed_new
    ;

target_typed_new
    : 'new' '(' argument_list? ')' object_or_collection_initializer?
    ;

object_or_collection_initializer
    : object_initializer
    | collection_initializer
    ;

// Source: §12.8.17.3 Object initializers
object_initializer
    : '{' member_initializer_list? '}'
    | '{' member_initializer_list ',' '}'
    ;

member_initializer_list
    : member_initializer (',' member_initializer)*
    ;

member_initializer
    : initializer_target '=' 'ref'? initializer_value
    ;

initializer_target
    : identifier
    | '[' argument_list ']'
    ;

initializer_value
    : 'ref'? expression
    | object_or_collection_initializer
    ;
```

```
// Source: §12.8.17.3.1 Collection initializers
collection_initializer
    : '{' element_initializer_list '}'
    | '{' element_initializer_list ',' '}'
    ;

element_initializer_list
    : element_initializer (',' element_initializer)*
    ;

element_initializer
    : non_assignment_expression
    | '{' expression_list '}'
    ;

expression_list
    : expression (',' expression)*
    ;

// Source: §12.8.17.4 Anonymous object creation expressions
anonymous_object_creation_expression
    : 'new' anonymous_object_initializer
    ;

anonymous_object_initializer
    : '{' member_declarator_list? '}'
    | '{' member_declarator_list ',' '}'
    ;

member_declarator_list
    : member_declarator (',' member_declarator)*
    ;

member_declarator
    : simple_name
    | member_access
    | null_conditional_projection_initializer
    | base_access
    | identifier '=' expression
    ;

// Source: §12.8.17.5 Array creation expressions
array_creation_expression
    : 'new' non_array_type '[' expression_list ']' rank_specifier*
      array_initializer?
    | 'new' array_type array_initializer
    | 'new' rank_specifier array_initializer
    ;
```

```
// Source: §12.8.17.6 Delegate creation expressions
delegate_creation_expression
    : 'new' delegate_type '(' expression ')'
    ;

// Source: §12.8.18 The typeof operator
typeof_expression
    : 'typeof' '(' type ')'
    | 'typeof' '(' unbound_type_name ')'
    | 'typeof' '(' 'void' ')'
    ;

unbound_type_name
    : identifier generic_dimension_specifier?
      ('.' identifier generic_dimension_specifier?)*
    | unbound_qualified_alias_member
      ('.' identifier generic_dimension_specifier?)*
    ;

unbound_qualified_alias_member
    : identifier '::' identifier generic_dimension_specifier?
    ;

generic_dimension_specifier
    : '<' comma* '>'
    ;

comma
    : ','
    ;


// Source: §12.8.19 The sizeof operator
sizeof_expression
    : 'sizeof' '(' unmanaged_type ')'
    ;

// Source: §12.8.20 The checked and unchecked operators
checked_expression
    : 'checked' '(' expression ')'
    ;

unchecked_expression
    : 'unchecked' '(' expression ')'
    ;

// Source: §12.8.21 Default value expressions
default_value_expression
```

```
    : explicitly_typed_default
    | default_literal
    ;

explicitly_typed_default
    : 'default' '(' type ')'
    ;

default_literal
    : 'default'
    ;

// Source: §12.8.22 Stack allocation
stackalloc_expression
    : 'stackalloc' unmanaged_type '[' expression ']'
    | 'stackalloc' unmanaged_type? '[' constant_expression? ']'
stackalloc_initializer
    ;

stackalloc_initializer
     : '{' stackalloc_initializer_element_list '}'
     ;

stackalloc_initializer_element_list
     : stackalloc_element_initializer (',' stackalloc_element_ini-
tializer)* ','?
     ;

stackalloc_element_initializer
    : expression
    ;

// Source: §12.8.23 The nameof operator
nameof_expression
    : 'nameof' '(' named_entity ')'
    ;

named_entity
    : named_entity_target ('.' identifier type_argument_list?)*
    ;

named_entity_target
    : simple_name
    | 'this'
    | 'base'
    | predefined_type
    | qualified_alias_member
    ;
```

```
// Source: §12.9.1 General
unary_expression
    : primary_expression
    | '+' unary_expression
    | '-' unary_expression
    | logical_negation_operator unary_expression
    | '~' unary_expression
    | '^' unary_expression
    | pre_increment_expression
    | pre_decrement_expression
    | cast_expression
    | await_expression
    | pointer_indirection_expression    // unsafe code support
    | addressof_expression              // unsafe code support
    ;

// Source: §12.9.7 Prefix increment and decrement operators
pre_increment_expression
    : '++' unary_expression
    ;

pre_decrement_expression
    : '--' unary_expression
    ;

// Source: §12.9.8 Cast expressions
cast_expression
    : '(' type ')' unary_expression
    ;

// Source: §12.9.9.1 General
await_expression
    : 'await' unary_expression
    ;

// Source: §12.10 With expressions
with_expression
    : switch_expression
    | switch_expression 'with' '{' member_initializer_list? '}'
    ;

// Source: §12.11 Range operator
range_expression
    : unary_expression
    | unary_expression? '..' unary_expression?
    ;

// Source: §12.12 Switch expression
switch_expression
```

```
    : range_expression
    | switch_expression 'switch' '{' switch_expression_arms? '}'
    ;

switch_expression_arms
    : switch_expression_arm (',' switch_expression_arm)* ','?
    ;

switch_expression_arm
    : pattern case_guard? '=>' switch_expression_arm_expression
    ;

switch_expression_arm_expression
    : expression
    ;

// Source: §12.13.1 General
multiplicative_expression
    : switch_expression
    | multiplicative_expression '*' switch_expression
    | multiplicative_expression '/' switch_expression
    | multiplicative_expression '%' switch_expression
    ;

additive_expression
    : multiplicative_expression
    | additive_expression '+' multiplicative_expression
    | additive_expression '-' multiplicative_expression
    ;

// Source: §12.14 Shift operators
shift_expression
    : additive_expression
    | shift_expression '<<' additive_expression
    | shift_expression right_shift additive_expression
    | shift_expression unsigned_right_shift additive_expression
    ;

// Source: §12.15.1 General
relational_expression
    : shift_expression
    | relational_expression '<' shift_expression
    | relational_expression '>' shift_expression
    | relational_expression '<=' shift_expression
    | relational_expression '>=' shift_expression
    | relational_expression 'is' type
    | relational_expression 'is' pattern
    | relational_expression 'as' type
    ;
```

```
equality_expression
    : relational_expression
    | equality_expression '==' relational_expression
    | equality_expression '!=' relational_expression
    ;

// Source: §12.16.1 General
and_expression
    : equality_expression
    | and_expression '&' equality_expression
    ;

exclusive_or_expression
    : and_expression
    | exclusive_or_expression '^' and_expression
    ;

inclusive_or_expression
    : exclusive_or_expression
    | inclusive_or_expression '|' exclusive_or_expression
    ;

// Source: §12.17.1 General
conditional_and_expression
    : inclusive_or_expression
    | conditional_and_expression '&&' inclusive_or_expression
    ;

conditional_or_expression
    : conditional_and_expression
    | conditional_or_expression '||' conditional_and_expression
    ;

// Source: §12.18 The null coalescing operator
null_coalescing_expression
    : conditional_or_expression
    | conditional_or_expression '??' null_coalescing_expression
    | throw_expression
    ;

// Source: §12.19 The throw expression operator
throw_expression
    : 'throw' null_coalescing_expression
    ;

// Source: §12.20 Declaration expressions
declaration_expression
    : 'scoped'? local_variable_type identifier
```

```
    ;

local_variable_type
    : type
    | 'var'
    ;

// Source: §12.21 Conditional operator
conditional_expression
    : null_coalescing_expression
    | null_coalescing_expression '?' expression ':' expression
    | null_coalescing_expression '?' 'ref' variable_reference ':'
      'ref' variable_reference
    ;

// Source: §12.22.1 General
lambda_expression
    : attributes? anonymous_function_modifier?
      (return_type | ref_kind ref_return_type)?
      anonymous_function_signature '=>' anonymous_function_body
    ;

anonymous_method_expression
    : anonymous_function_modifier? 'delegate' explicit_anony-
mous_function_signature? block
    ;

anonymous_function_modifier
    : 'async' 'static'?
    | 'static' 'async'?
    ;

anonymous_function_signature
    : explicit_anonymous_function_signature
    | implicit_anonymous_function_signature
    ;

explicit_anonymous_function_signature
    : '(' explicit_anonymous_function_parameter_list? ')'
    ;

explicit_anonymous_function_parameter_list
    : explicit_anonymous_function_parameter
      (',' explicit_anonymous_function_parameter)*
    ;

explicit_anonymous_function_parameter
    : attributes? 'scoped'? anonymous_function_parameter_modifier?
type identifier
```

```
    ;

anonymous_function_parameter_modifier
    : 'ref'
    | 'out'
    | 'in'
    ;

implicit_anonymous_function_signature
    : '(' implicit_anonymous_function_parameter_list? ')'
    | implicit_anonymous_function_parameter
    ;

implicit_anonymous_function_parameter_list
    : implicit_anonymous_function_parameter
      (',' implicit_anonymous_function_parameter)*
    ;

implicit_anonymous_function_parameter
    : attributes? identifier
    ;

anonymous_function_body
    : null_conditional_invocation_expression
    | expression
    | 'ref' variable_reference
    | block
    ;

// Source: §12.23.1 General
query_expression
    : from_clause query_body
    ;

from_clause
    : 'from' type? identifier 'in' expression
    ;

query_body
    : query_body_clause* select_or_group_clause query_continuation?
    ;

query_body_clause
    : from_clause
    | let_clause
    | where_clause
    | join_clause
    | join_into_clause
    | orderby_clause
```

```
    ;

let_clause
    : 'let' identifier '=' expression
    ;

where_clause
    : 'where' boolean_expression
    ;

join_clause
    : 'join' type? identifier 'in' expression 'on' expression
      'equals' expression
    ;

join_into_clause
    : 'join' type? identifier 'in' expression 'on' expression
      'equals' expression 'into' identifier
    ;

orderby_clause
    : 'orderby' orderings
    ;

orderings
    : ordering (',' ordering)*
    ;

ordering
    : expression ordering_direction?
    ;

ordering_direction
    : 'ascending'
    | 'descending'
    ;

select_or_group_clause
    : select_clause
    | group_clause
    ;

select_clause
    : 'select' expression
    ;

group_clause
    : 'group' expression 'by' expression
    ;
```

```
query_continuation
    : 'into' identifier query_body
    ;

// Source: §12.24.1 General
assignment
    : deconstructing_assignment
    | simple_assignment
    | compound_assignment
    | ref_assignment
    ;

// Source: §12.24.2 Simple assignment
simple_assignment
    : unary_expression '=' expression
    ;

// Source: §12.24.3.1 General
deconstructing_assignment
    : deconstructor '=' expression
    ;

deconstructor
    : '(' deconstructor_element (',' deconstructor_element)+ ')'
    | abridged_deconstructor
    ;

deconstructor_element
    : deconstructor
    | declaration_expression
    | discard_token
    | variable_reference
    ;

// Source: §12.24.3.2 Abridged deconstructors
abridged_deconstructor
    : 'var' abridged_deconstructor_elements
    ;

abridged_deconstructor_elements
    : '(' abridged_deconstructor_element (',' abridged_deconstruc-
tor_element)+ ')'
    ;

abridged_deconstructor_element
    : identifier
    | abridged_deconstructor_elements
    ;
```

```
// Source: §12.24.4 Ref assignment
ref_assignment
    : unary_expression '=' 'ref' expression
    ;

// Source: §12.24.5 Compound assignment
compound_assignment
    : unary_expression compound_assignment_operator expression
    ;

compound_assignment_operator
    : '+=' | '-=' | '*=' | '/=' | '%=' | '&=' | '|=' | '^=' | '<<='
| '??='
    | right_shift_assignment
    ;

// Source: §12.25 Expression
expression
    : non_assignment_expression
    | assignment
    ;

non_assignment_expression
    : conditional_expression
    | lambda_expression
    | query_expression
    ;

// Source: §12.26 Constant expressions
constant_expression
    : conditional_expression
    ;

// Source: §12.27 Boolean expressions
boolean_expression
    : expression
    ;

// Source: §13.1 General
statement
    : labeled_statement
    | declaration_statement
    | embedded_statement
    ;

embedded_statement
    : block
    | empty_statement
```

```
    | expression_statement
    | selection_statement
    | iteration_statement
    | jump_statement
    | try_statement
    | checked_statement
    | unchecked_statement
    | lock_statement
    | using_statement
    | yield_statement
    | unsafe_statement   // unsafe code support
    | fixed_statement    // unsafe code support
    ;

// Source: §13.3.1 General
block
    : '{' statement_list? '}'
    ;

// Source: §13.3.2 Statement lists
statement_list
    : statement+
    ;

// Source: §13.4 The empty statement
empty_statement
    : ';'
    ;

// Source: §13.5 Labeled statements
labeled_statement
    : identifier ':' statement
    ;

// Source: §13.6.1 General
declaration_statement
    : local_variable_declaration ';'
    | local_constant_declaration ';'
    | local_function_declaration
    | local_using_declaration
    ;

// Source: §13.6.2.1 General
local_variable_declaration
    : implicitly_typed_local_variable_declaration
    | explicitly_typed_local_variable_declaration
    | explicitly_typed_ref_local_variable_declaration
    ;
```

```
// Source: §13.6.2.2 Implicitly typed local variable declarations
implicitly_typed_local_variable_declaration
    : 'scoped'? 'var' implicitly_typed_local_variable_declarator
    | 'scoped'? ref_kind 'var' ref_local_variable_declarator
    ;

implicitly_typed_local_variable_declarator
    : identifier '=' expression
    ;

// Source: §13.6.2.3 Explicitly typed local variable declarations
explicitly_typed_local_variable_declaration
    : 'scoped'? type explicitly_typed_local_variable_declarators
    ;

explicitly_typed_local_variable_declarators
    : explicitly_typed_local_variable_declarator
      (',' explicitly_typed_local_variable_declarator)*
    ;

explicitly_typed_local_variable_declarator
    : identifier ('=' local_variable_initializer)?
    ;

local_variable_initializer
    : expression
    | array_initializer
    ;

// Source: §13.6.2.4 Explicitly typed ref local variable declara-
tions
explicitly_typed_ref_local_variable_declaration
    : 'scoped'? ref_kind type ref_local_variable_declarators
    ;

ref_local_variable_declarators
    : ref_local_variable_declarator (',' ref_local_variable_declara-
tor)*
    ;

ref_local_variable_declarator
    : identifier '=' 'ref' variable_reference
    ;

// Source: §13.6.3 Local constant declarations
local_constant_declaration
    : 'const' type constant_declarators
    ;
```

```
constant_declarators
    : constant_declarator (',' constant_declarator)*
    ;

constant_declarator
    : identifier '=' constant_expression
    ;

// Source: §13.6.4 Local function declarations
local_function_declaration
    : attributes? local_function_modifier* return_type local_func-
tion_header
      local_function_body
    | attributes? ref_local_function_modifier* ref_kind ref_re-
turn_type
      local_function_header ref_local_function_body
    ;

local_function_header
    : identifier '(' parameter_list? ')'
    | identifier type_parameter_list '(' parameter_list? ')'
      type_parameter_constraints_clause*
    ;

local_function_modifier
    : ref_local_function_modifier
    | 'async'
    ;

ref_local_function_modifier
    : 'static'
    | 'extern'
    | unsafe_modifier   // unsafe code support
    ;

local_function_body
    : block
    | '=>' null_conditional_invocation_expression ';'
    | '=>' expression ';'
    | ';'
    ;

ref_local_function_body
    : block
    | '=>' 'ref' variable_reference ';'
    | ';'
    ;

// Source: §13.7 Expression statements
```

```
expression_statement
    : statement_expression ';'
    ;

statement_expression
    : null_conditional_invocation_expression
    | invocation_expression
    | object_creation_expression
    | assignment
    | post_increment_expression
    | post_decrement_expression
    | pre_increment_expression
    | pre_decrement_expression
    | await_expression
    ;

// Source: §13.8.1 General
selection_statement
    : if_statement
    | switch_statement
    ;

// Source: §13.8.2 The if statement
if_statement
    : 'if' '(' boolean_expression ')' embedded_statement
    | 'if' '(' boolean_expression ')' embedded_statement
      'else' embedded_statement
    ;

// Source: §13.8.3 The switch statement
switch_statement
    : 'switch' selector_expression switch_block
    ;

selector_expression
    : '(' expression ')'
    | tuple_literal
    ;

switch_block
    : '{' switch_section* '}'
    ;

switch_section
    : switch_label+ statement_list
    ;

switch_label
    : 'case' pattern case_guard?  ':'
```

```
    | 'default' ':'
    ;

case_guard
    : 'when' null_coalescing_expression
    ;

// Source: §13.9.1 General
iteration_statement
    : while_statement
    | do_statement
    | for_statement
    | foreach_statement
    ;

// Source: §13.9.2 The while statement
while_statement
    : 'while' '(' boolean_expression ')' embedded_statement
    ;

// Source: §13.9.3 The do statement
do_statement
    : 'do' embedded_statement 'while' '(' boolean_expression ')' ';'
    ;

// Source: §13.9.4 The for statement
for_statement
    : 'for' '(' for_initializer? ';' for_condition? ';' for_itera-
tor? ')'
      embedded_statement
    ;

for_initializer
    : local_variable_declaration
    | statement_expression_list
    ;

for_condition
    : boolean_expression
    ;

for_iterator
    : statement_expression_list
    ;

statement_expression_list
    : statement_expression (',' statement_expression)*
    ;
```

```
// Source: §13.9.5.1 General
foreach_statement
    : // synchronous foreach
      'foreach' '(' 'scoped'? ref_kind? local_variable_type identi-
fier 'in' expression ')'
          embedded_statement
    | // asynchronous foreach
      'await' 'foreach' '(' local_variable_type identifier 'in' ex-
pression ')'
          embedded_statement
    | // deconstructing foreach
      'await'? 'foreach' '(' deconstructor 'in' expression ')'
          embedded_statement
    ;

// Source: §13.10.1 General
jump_statement
    : break_statement
    | continue_statement
    | goto_statement
    | return_statement
    | throw_statement
    ;

// Source: §13.10.2 The break statement
break_statement
    : 'break' ';'
    ;

// Source: §13.10.3 The continue statement
continue_statement
    : 'continue' ';'
    ;

// Source: §13.10.4 The goto statement
goto_statement
    : 'goto' identifier ';'
    | 'goto' 'case' constant_expression ';'
    | 'goto' 'default' ';'
    ;

// Source: §13.10.5 The return statement
return_statement
    : 'return' ';'
    | 'return' expression ';'
    | 'return' 'ref' variable_reference ';'
    ;

// Source: §13.10.6 The throw statement
```

```
throw_statement
    : 'throw' expression? ';'
    ;

// Source: §13.11 The try statement
try_statement
    : 'try' block catch_clauses
    | 'try' block catch_clauses? finally_clause
    ;

catch_clauses
    : specific_catch_clause+
    | specific_catch_clause* general_catch_clause
    ;

specific_catch_clause
    : 'catch' exception_specifier exception_filter? block
    | 'catch' exception_filter block
    ;

exception_specifier
    : '(' type identifier? ')'
    ;

exception_filter
    : 'when' '(' boolean_expression ')'
    ;

general_catch_clause
    : 'catch' block
    ;

finally_clause
    : 'finally' block
    ;

// Source: §13.12 The checked and unchecked statements
checked_statement
    : 'checked' block
    ;

unchecked_statement
    : 'unchecked' block
    ;

// Source: §13.13 The lock statement
lock_statement
    : 'lock' '(' expression ')' embedded_statement
    ;
```

```
// Source: §13.14.1 General
using_statement
    : 'await'? 'using' '(' resource_acquisition ')' embedded_state-
ment
    ;

resource_acquisition
    : non_ref_local_variable_declaration
    | expression
    ;

non_ref_local_variable_declaration
    : implicitly_typed_local_variable_declaration
    | explicitly_typed_local_variable_declaration
    ;

// Source: §13.14.2 Using declaration
local_using_declaration
    : 'await'? 'using' non_ref_local_variable_declaration ';' state-
ment_list?
    ;

// Source: §13.15 The yield statement
yield_statement
    : 'yield' 'return' expression ';'
    | 'yield' 'break' ';'
    ;

// Source: §14.2 Compilation units
compilation_unit
    : extern_alias_directive* using_directive* global_attributes?
compilation_unit_body
    ;

compilation_unit_body
    : statement_list* namespace_member_declaration*
    | file_scoped_namespace_declaration
    ;

// Source: §14.3 Namespace declarations
namespace_declaration
    : 'namespace' qualified_identifier namespace_body ';'?
    ;

file_scoped_namespace_declaration
    : 'namespace' qualified_identifier ';' extern_alias_directive*
using_directive*
      type_declaration*
```

```
    ;

qualified_identifier
    : identifier ('.' identifier)*
    ;

namespace_body
    : '{' extern_alias_directive* using_directive*
      namespace_member_declaration* '}'
    ;

// Source: §14.4 Extern alias directives
extern_alias_directive
    : 'extern' 'alias' identifier ';'
    ;

// Source: §14.5.1 General
global_using_directive
    : global_using_alias_directive
    | global_using_namespace_directive
    | global_using_static_directive
    ;

// Source: §14.5.2 Global using alias directives
global_using_alias_directive
    : 'global' using_alias_directive
    ;

// Source: §14.5.3 Global using namespace directives
global_using_namespace_directive
    : 'global' using_namespace_directive
    ;

// Source: §14.5.4 Global using static directives
global_using_static_directive
    : 'global' using_static_directive
    ;

// Source: §14.6.1 General
using_directive
    : using_alias_directive
    | using_namespace_directive
    | using_static_directive
    ;

// Source: §14.6.2 Using alias directives
using_alias_directive
    : 'using' identifier '=' namespace_or_type_name ';'
    ;
```

```
// Source: §14.6.3 Using namespace directives
using_namespace_directive
    : 'using' namespace_name ';'
    ;

// Source: §14.6.4 Using static directives
using_static_directive
    : 'using' 'static' type_name ';'
    ;

// Source: §14.7 Namespace member declarations
namespace_member_declaration
    : namespace_declaration
    | type_declaration
    ;

// Source: §14.8 Type declarations
type_declaration
    : class_declaration
    | struct_declaration
    | interface_declaration
    | enum_declaration
    | delegate_declaration
    ;

// Source: §14.9.1 General
qualified_alias_member
    : identifier '::' identifier type_argument_list?
    ;

// Source: §15.2.1 General
class_declaration
    : attributes? class_modifier* 'partial'? class_tag identifier
        type_parameter_list? delimited_parameter_list? class_base?
        type_parameter_constraints_clause* class_body
    ;

class_tag
    : 'record'? 'class'
    | 'record'
    ;

// Source: §15.2.2.1 General
class_modifier
    : 'new'
    | 'public'
    | 'protected'
    | 'internal'
```

```
    | 'private'
    | 'abstract'
    | 'sealed'
    | 'static'
    | 'file'
    | unsafe_modifier   // unsafe code support
    ;

// Source: §15.2.3 Type parameters
type_parameter_list
    : '<' decorated_type_parameter (',' decorated_type_parameter)*
'>'
    ;

decorated_type_parameter
    : attributes? type_parameter
    ;

// Source: §15.2.4.1 General
class_base
    : ':' class_type base_argument_list?
    | ':' interface_type_list
    | ':' class_type base_argument_list? ',' interface_type_list
    ;

base_argument_list
    : '(' argument_list? ')'
    ;

interface_type_list
    : interface_type (',' interface_type)*
    ;

// Source: §15.2.5 Type parameter constraints
type_parameter_constraints_clause
    : 'where' type_parameter ':' type_parameter_constraints
    ;

type_parameter_constraints
    : primary_constraint (',' secondary_constraints)? (',' construc-
tor_constraint)?
    | secondary_constraints (',' constructor_constraint)?
    | constructor_constraint
    ;

primary_constraint
    : class_type nullable_type_annotation?
    | 'class' nullable_type_annotation?
    | 'struct'
```

```
    | 'notnull'
    | 'unmanaged'
    | 'default'
    ;

secondary_constraint
    : interface_type nullable_type_annotation?
    | type_parameter nullable_type_annotation?
    ;

secondary_constraints
    : secondary_constraint (',' secondary_constraint)*
    ;

constructor_constraint
    : 'new' '(' ')'
    ;

// Source: §15.2.6 Class body
class_body
    : '{' class_member_declaration* '}' ';'?
    | ';'
    ;

// Source: §15.3.1 General
class_member_declaration
    : constant_declaration
    | field_declaration
    | method_declaration
    | property_declaration
    | event_declaration
    | indexer_declaration
    | operator_declaration
    | constructor_declaration
    | finalizer_declaration
    | static_constructor_declaration
    | type_declaration
    ;

// Source: §15.4 Constants
constant_declaration
    : attributes? constant_modifier* 'const' type constant_declara-
tors ';'
    ;

constant_modifier
    : 'new'
    | 'public'
    | 'protected'
```

```
    | 'internal'
    | 'private'
    ;

// Source: §15.5.1 General
field_declaration
    : attributes? field_modifier* type variable_declarators ';'
    ;

field_modifier
    : 'new'
    | 'public'
    | 'protected'
    | 'internal'
    | 'private'
    | 'static'
    | 'readonly'
    | 'volatile'
    | 'required'
    | unsafe_modifier   // unsafe code support
    ;

variable_declarators
    : variable_declarator (',' variable_declarator)*
    ;

variable_declarator
    : identifier ('=' variable_initializer)?
    ;

// Source: §15.6.1 General
method_declaration
    : attributes? method_modifiers return_type method_header
method_body
    | attributes? ref_method_modifiers ref_kind ref_return_type
method_header
      ref_method_body
    ;

method_modifiers
    : method_modifier* 'partial'?
    ;

ref_kind
    : 'ref'
    | 'ref' 'readonly'
    ;

ref_method_modifiers
```

```
    : ref_method_modifier* 'partial'?
    ;

method_header
    : member_name '(' parameter_list? ')'
    | member_name type_parameter_list '(' parameter_list? ')'
      type_parameter_constraints_clause*
    ;

method_modifier
    : ref_method_modifier
    | 'async'
    ;

ref_method_modifier
    : 'new'
    | 'public'
    | 'protected'
    | 'internal'
    | 'private'
    | 'static'
    | 'virtual'
    | 'sealed'
    | 'override'
    | 'abstract'
    | 'extern'
    | 'readonly'        // struct members only
    | unsafe_modifier   // unsafe code support
    ;

return_type
    : ref_return_type
    | 'void'
    ;

ref_return_type
    : type
    ;

member_name
    : identifier
    | interface_type '.' identifier
    ;

method_body
    : block
    | '=>' null_conditional_invocation_expression ';'
    | '=>' expression ';'
    | ';'
```

```
    ;

ref_method_body
    : block
    | '=>' 'ref' variable_reference ';'
    | ';'
    ;

// Source: §15.6.2.1 General
delimited_parameter_list
    : '(' parameter_list? ')'
    ;

parameter_list
    : fixed_parameters
    | fixed_parameters ',' parameter_array
    | parameter_array
    ;

fixed_parameters
    : fixed_parameter (',' fixed_parameter)*
    ;

fixed_parameter
    : attributes? parameter_modifier? type identifier default_argu-
ment?
    ;

default_argument
    : '=' expression
    ;

parameter_modifier
    : parameter_mode_modifier
    | 'this' 'scoped'? parameter_mode_modifier?
    | 'scoped'? parameter_mode_modifier? 'this'
    | 'scoped' parameter_mode_modifier?
    ;

parameter_mode_modifier
    : 'ref'
    | 'out'
    | 'in'
    ;

parameter_array
    : attributes? 'params' array_type identifier
    ;
```

```
// Source: §15.7.1 General
property_declaration
    : attributes? property_modifier* type member_name property_body
    | attributes? property_modifier* ref_kind type member_name
ref_property_body
    ;

property_modifier
    : 'new'
    | 'public'
    | 'protected'
    | 'internal'
    | 'private'
    | 'static'
    | 'virtual'
    | 'sealed'
    | 'override'
    | 'abstract'
    | 'extern'
    | 'readonly'        // struct members only
    | 'required'
    | unsafe_modifier   // unsafe code support
    ;

property_body
    : '{' accessor_declarations '}' property_initializer?
    | '=>' expression ';'
    ;

property_initializer
    : '=' variable_initializer ';'
    ;

ref_property_body
    : '{' ref_get_accessor_declaration '}'
    | '=>' 'ref' variable_reference ';'
    ;

// Source: §15.7.3.1 General
accessor_declarations
    : get_accessor_declaration
      (set_accessor_declaration | init_accessor_declaration)?
    | (set_accessor_declaration | init_accessor_declaration)
      get_accessor_declaration?
    ;

get_accessor_declaration
    : attributes? accessor_modifier? 'get' accessor_body
    ;
```

```
set_accessor_declaration
    : attributes? accessor_modifier? 'set' accessor_body
    ;

init_accessor_declaration
    : attributes? accessor_modifier? 'init' accessor_body
    ;

accessor_modifier
    : 'protected'
    | 'internal'
    | 'private'
    | 'protected' 'internal'
    | 'internal' 'protected'
    | 'protected' 'private'
    | 'private' 'protected'
    | 'readonly'        // struct members only
    ;

accessor_body
    : block
    | '=>' expression ';'
    | ';'
    ;

ref_get_accessor_declaration
    : attributes? accessor_modifier? 'get' ref_accessor_body
    ;

ref_accessor_body
    : block
    | '=>' 'ref' variable_reference ';'
    | ';'
    ;

// Source: §15.8.1 General
event_declaration
    : attributes? event_modifier* 'event' type variable_declarators
';'
    | attributes? event_modifier* 'event' type member_name
        '{' event_accessor_declarations '}'
    ;

event_modifier
    : 'new'
    | 'public'
    | 'protected'
    | 'internal'
```

```
    | 'private'
    | 'static'
    | 'virtual'
    | 'sealed'
    | 'override'
    | 'abstract'
    | 'extern'
    | 'readonly'        // struct members only
    | unsafe_modifier   // unsafe code support
    ;

event_accessor_declarations
    : add_accessor_declaration remove_accessor_declaration
    | remove_accessor_declaration add_accessor_declaration
    ;

add_accessor_declaration
    : attributes? 'add' block
    ;

remove_accessor_declaration
    : attributes? 'remove' block
    ;

// Source: §15.9.1 General
indexer_declaration
    : attributes? indexer_modifier* indexer_declarator indexer_body
    | attributes? indexer_modifier* ref_kind indexer_declarator
ref_indexer_body
    ;

indexer_modifier
    : 'new'
    | 'public'
    | 'protected'
    | 'internal'
    | 'private'
    | 'virtual'
    | 'sealed'
    | 'override'
    | 'abstract'
    | 'extern'
    | 'readonly'        // struct members only
    | unsafe_modifier   // unsafe code support
    ;

indexer_declarator
    : type 'this' '[' parameter_list ']'
    | type interface_type '.' 'this' '[' parameter_list ']'
```

```
    ;

indexer_body
    : '{' accessor_declarations '}'
    | '=>' expression ';'
    ;

ref_indexer_body
    : '{' ref_get_accessor_declaration '}'
    | '=>' 'ref' variable_reference ';'
    ;

// Source: §15.10.1 General
operator_declaration
    : attributes? operator_modifier+ operator_declarator opera-
tor_body
    ;

operator_modifier
    : 'public'
    | 'static'
    | 'extern'
    | unsafe_modifier   // unsafe code support
    ;

operator_declarator
    : unary_operator_declarator
    | binary_operator_declarator
    | conversion_operator_declarator
    ;

unary_operator_declarator
    : type 'operator' overloadable_unary_operator '(' fixed_parame-
ter ')'
    ;

logical_negation_operator
    : '!'
    ;

overloadable_unary_operator
    : '+' | 'checked'? '-' | '!' | '~' | 'checked'? '++' |
'checked'? '--' | 'true' | 'false'
    ;

binary_operator_declarator
    : type 'operator' overloadable_binary_operator
        '(' fixed_parameter ',' fixed_parameter ')'
    ;
```

```
overloadable_binary_operator
    : 'checked'? '+'  | 'checked'? '-'  | 'checked'? '*'  |
'checked'? '/'  | '%'  | '&' | '|' | '^'  | '<<'
      | right_shift | unsigned_right_shift | '==' | '!=' | '>' | '<'
| '>=' | '<='
    ;

conversion_operator_declarator
    : 'implicit' 'operator' type '(' fixed_parameter ')'
    | 'explicit' 'operator' 'checked'? type '(' fixed_parameter ')'
    ;

operator_body
    : block
    | '=>' expression ';'
    | ';'
    ;

// Source: §15.11.1 General
constructor_declaration
    : attributes? constructor_modifier* constructor_declarator con-
structor_body
    ;

constructor_modifier
    : 'public'
    | 'protected'
    | 'internal'
    | 'private'
    | 'extern'
    | unsafe_modifier   // unsafe code support
    ;

constructor_declarator
    : identifier '(' parameter_list? ')' constructor_initializer?
    ;

constructor_initializer
    : ':' 'base' '(' argument_list? ')'
    | ':' 'this' '(' argument_list? ')'
    ;

constructor_body
    : block
    | '=>' expression ';'
    | ';'
    ;
```

```
// Source: §15.12 Static constructors
static_constructor_declaration
    : attributes? static_constructor_modifiers identifier '(' ')'
      static_constructor_body
    ;

static_constructor_modifiers
    : 'static'
    | 'static' 'extern' unsafe_modifier?
    | 'static' unsafe_modifier 'extern'?
    | 'extern' 'static' unsafe_modifier?
    | 'extern' unsafe_modifier 'static'
    | unsafe_modifier 'static' 'extern'?
    | unsafe_modifier 'extern' 'static'
    ;

static_constructor_body
    : block
    | '=>' expression ';'
    | ';'
    ;

// Source: §15.13 Finalizers
finalizer_declaration
    : attributes? '~' identifier '(' ')' finalizer_body
    | attributes? 'extern' unsafe_modifier? '~' identifier '(' ')'
      finalizer_body
    | attributes? unsafe_modifier 'extern'? '~' identifier '(' ')'
      finalizer_body
    ;

finalizer_body
    : block
    | '=>' expression ';'
    | ';'
    ;

// Source: §16.2.1 General
struct_declaration
    : non_record_struct_declaration
    | record_struct_declaration
    ;

non_record_struct_declaration
    : attributes? struct_modifier* 'ref'? 'partial'? 'struct'
      identifier type_parameter_list? struct_interfaces?
      type_parameter_constraints_clause* struct_body ';'?
    ;
```

```
record_struct_declaration
    : attributes? struct_modifier* 'partial'? 'record' 'struct'
      identifier type_parameter_list? delimited_parameter_list?
struct_interfaces?
      type_parameter_constraints_clause* record_struct_body
    ;

record_struct_body
    : struct_body ';'?
    | ';'
    ;

// Source: §16.2.2 Struct modifiers
struct_modifier
    : 'new'
    | 'public'
    | 'protected'
    | 'internal'
    | 'private'
    | 'readonly'
    | 'file'
    | unsafe_modifier   // unsafe code support
    ;

// Source: §16.2.5 Struct interfaces
struct_interfaces
    : ':' interface_type_list
    ;

// Source: §16.2.6 Struct body
struct_body
    : '{' struct_member_declaration* '}'
    ;

// Source: §16.3.1 General
struct_member_declaration
    : constant_declaration
    | struct_field_declaration
    | method_declaration
    | property_declaration
    | event_declaration
    | indexer_declaration
    | operator_declaration
    | constructor_declaration
    | static_constructor_declaration
    | type_declaration
    | fixed_size_buffer_declaration   // unsafe code support
    ;
```

```
// Source: §16.5.8.2 Ref fields
struct_field_declaration
    : attributes? field_modifier* ('readonly'? 'ref' 'readonly'?)?
type
      variable_declarators ';'
    ;

// Source: §17.7 Array initializers
array_initializer
    : '{' variable_initializer_list? '}'
    | '{' variable_initializer_list ',' '}'
    ;

variable_initializer_list
    : variable_initializer (',' variable_initializer)*
    ;

variable_initializer
    : expression
    | array_initializer
    ;

// Source: §19.2.1 General
interface_declaration
    : attributes? interface_modifier* 'partial'? 'interface'
      identifier variant_type_parameter_list? interface_base?
      type_parameter_constraints_clause* interface_body ';'?
    ;

// Source: §19.2.2 Interface modifiers
interface_modifier
    : 'new'
    | 'public'
    | 'protected'
    | 'internal'
    | 'private'
    | 'file'
    | unsafe_modifier   // unsafe code support
    ;

// Source: §19.2.3.1 General
variant_type_parameter_list
    : '<' variant_type_parameter (',' variant_type_parameter)* '>'
    ;

variant_type_parameter
    : attributes? variance_annotation? type_parameter
    ;
```

```
variance_annotation
    : 'in'
    | 'out'
    ;

// Source: §19.2.4 Base interfaces
interface_base
    : ':' interface_type_list
    ;

// Source: §19.3 Interface body
interface_body
    : '{' interface_member_declaration* '}'
    ;

// Source: §19.4.1 General
interface_member_declaration
    : constant_declaration
    | field_declaration
    | method_declaration
    | property_declaration
    | event_declaration
    | indexer_declaration
    | static_constructor_declaration
    | operator_declaration
    | type_declaration
    ;

// Source: §20.2 Enum declarations
enum_declaration
    : attributes? enum_modifier* 'enum' identifier enum_base?
enum_body ';'?
    ;

enum_base
    : ':' integral_type
    | ':' integral_type_name
    ;

integral_type_name
    : type_name // Shall resolve to an integral type other than char
    ;

enum_body
    : '{' enum_member_declarations? '}'
    | '{' enum_member_declarations ',' '}'
    ;

// Source: §20.3 Enum modifiers
```

```
enum_modifier
    : 'new'
    | 'public'
    | 'protected'
    | 'internal'
    | 'private'
    | 'file'
    ;

// Source: §20.4 Enum members
enum_member_declarations
    : enum_member_declaration (',' enum_member_declaration)*
    ;

// Source: §20.4 Enum members
enum_member_declaration
    : attributes? identifier ('=' constant_expression)?
    ;

// Source: §21.2 Delegate declarations
delegate_declaration
    : attributes? delegate_modifier* 'delegate' return_type dele-
gate_header
    | attributes? delegate_modifier* 'delegate' ref_kind ref_re-
turn_type
      delegate_header
    ;

delegate_header
    : identifier '(' parameter_list? ')' ';'
    | identifier variant_type_parameter_list '(' parameter_list? ')'
      type_parameter_constraints_clause* ';'
    ;

delegate_modifier
    : 'new'
    | 'public'
    | 'protected'
    | 'internal'
    | 'private'
    | 'file'
    | unsafe_modifier   // unsafe code support
    ;

// Source: §23.3 Attribute specification
global_attributes
    : global_attribute_section+
    ;
```

```
global_attribute_section
    : '[' global_attribute_target_specifier attribute_list ']'
    ;

global_attribute_target_specifier
    : global_attribute_target ':'
    ;

global_attribute_target
    : identifier
    ;

attributes
    : attribute_section+
    ;

attribute_section
    : '[' attribute_target_specifier? attribute_list ']'
    ;

attribute_target_specifier
    : attribute_target ':'
    ;

attribute_target
    : identifier
    | keyword
    ;

attribute_list
    : attribute (',' attribute)* ','?
    ;

attribute
    : attribute_name attribute_arguments?
    ;

attribute_name
    : type_name
    ;

attribute_arguments
    : '(' ')'
    | '(' positional_argument_list (',' named_argument_list)? ')'
    | '(' named_argument_list ')'
    ;

positional_argument_list
    : positional_argument (',' positional_argument)*
```

```
    ;

positional_argument
    : argument_name? attribute_argument_expression
    ;

named_argument_list
    : named_argument (','  named_argument)*
    ;

named_argument
    : identifier '=' attribute_argument_expression
    ;

attribute_argument_expression
    : non_assignment_expression
    ;
```

A.4 Grammar extensions for unsafe code

ANTLR

```
// Source: §24.2 Unsafe contexts
unsafe_modifier
    : 'unsafe'
    ;

unsafe_statement
    : 'unsafe' block
    ;

// Source: §24.3.1 General
pointer_type
    : dataptr_type
    | funcptr_type
    | voidptr_type
    ;

// Source: §24.3.2 Data pointers
dataptr_type
    : value_type ('*')+
    | funcptr_type ('*')+
    | voidptr_type ('*')+
    ;
```

```
// Source: §24.3.3 Function pointers
funcptr_type
    : 'delegate' '*' calling_convention_specifier?
      '<' funcptr_parameter_list funcptr_return_type '>'
    ;

calling_convention_specifier
    : 'managed'
    | 'unmanaged' ('[' unmanaged_calling_convention ']')?
    ;

unmanaged_calling_convention
    : 'Cdecl'
    | 'Stdcall'
    | 'Thiscall'
    | 'Fastcall'
    | identifier (',' identifier)*
    ;

funcptr_parameter_list
    : (funcptr_parameter ',')*
    ;

funcptr_parameter
    : parameter_mode_modifier? type
    ;

funcptr_return_type
    : ref_kind? return_type
    ;

// Source: §24.3.4 Void pointers
voidptr_type
    : 'void' '*'
    ;

// Source: §24.6.2 Pointer indirection
pointer_indirection_expression
    : '*' unary_expression
    ;

// Source: §24.6.3 Pointer member access
pointer_member_access
    : primary_expression '->' identifier type_argument_list?
    ;

// Source: §24.6.4 Pointer element access
pointer_element_access
    : primary_expression '[' expression ']'
```

```
    ;

// Source: §24.6.5 The address-of operator
addressof_expression
    : '&' unary_expression
    ;

// Source: §24.7 The fixed statement
fixed_statement
    : 'fixed' '(' pointer_type fixed_pointer_declarators ')' embed-
ded_statement
    ;

fixed_pointer_declarators
    : fixed_pointer_declarator (','  fixed_pointer_declarator)*
    ;

fixed_pointer_declarator
    : identifier '=' fixed_pointer_initializer
    ;

fixed_pointer_initializer
    : '&' variable_reference
    | expression
    ;

// Source: §24.8.2 Fixed-size buffer declarations
fixed_size_buffer_declaration
    : attributes? fixed_size_buffer_modifier* 'fixed' buffer_ele-
ment_type
      fixed_size_buffer_declarators ';'
    ;

fixed_size_buffer_modifier
    : 'new'
    | 'public'
    | 'internal'
    | 'private'
    | 'unsafe'
    ;

buffer_element_type
    : type
    ;

fixed_size_buffer_declarators
    : fixed_size_buffer_declarator (',' fixed_size_buffer_declara-
tor)*
    ;
```

```
fixed_size_buffer_declarator
    : identifier '[' constant_expression ']'
    ;
```

End of informative text.

Last updated on 09/12/2025